

SUBSTR: specify the positions

Make a hidden translation choice visible

01 / THE SAS EXAMPLE

```
data test;
  length text $20 first_three $3
         middle $4 from_seventh $14;
  input text :$20.;
  first_three = substr(text, 1, 3);
  middle = substr(text, 5, 4);
  from_seventh = substr(text, 7);
  datalines;
ABCDEFGHIJ
HelloWorld
SAS-to-Python
;
run;

proc print data=test noobs;
run;
```

WHAT YOU WILL PRACTICE

Use concrete examples to check a translation rule instead of judging code by appearance.

Extract the beginning, middle, and remainder of each string. SAS positions start at 1; Python slice positions start at 0. Check each result character by character.

EXPECTED RESULT

Row	first_three	middle	from_seventh
1	ABC	EFGH	GHIJ
2	Hel	oWor	orld
3	SAS	to-P	-Python

The table hides trailing storage blanks for readability. Stored widths are 20 for text and 3, 4, 14 for the extracted columns.

Expected values checked in Python; SAS execution pending.

ATTACH SOURCE OR PASTE IT AFTER THE PROMPT. START FRESH EACH TIME.

Prompt A: leave choices open

High degrees of freedom / Notice the decisions the model makes for you.

COPY THIS PROMPT + THE SAS EXAMPLE

Convert this SAS code to Python.

Show code in this response. Do not create or run files, or read existing Python files.

LOOK AT THE RESPONSE

Which library and method did it choose? Did it preserve the expected behavior? Compare the result with page 1 before accepting it.

TRY IT / THREE FRESH ATTEMPTS

Run A three times with the same SAS example and setup. Save each first response before asking follow-up questions. Record agreement as well as differences.

Bedrock A/B trials for this example are pending.

WHY CHOICES REMAIN OPEN

- 1 The code has the rule**
The model can infer each start position and requested character count from the SAS calls.
- 2 The method is open**
Python slices, helper functions, or pandas methods can produce the same correct strings.
- 3 Appearance is a weak check**
A one-character shift can still look plausible. Compare outputs with the actual positions.

ANTI-PATTERN TO AVOID

Anti-pattern: saying 'looks right' after checking only ABC. A shifted start elsewhere may go unnoticed. Count the characters in HelloWorld and SAS-to-Python before accepting the conversion.

Prompt B: make the choices explicit

Low degrees of freedom / State the behavior, then ask for a check.

COPY THIS PROMPT + THE SAME SAS EXAMPLE

```
Convert this SAS code to Python using pandas.
- Build a DataFrame with text in the supplied row order.
- Right-pad text with spaces to width 20 before slicing.
- Use .str.slice(). Subtract 1 from the SAS start position.
  Python stop = adjusted start + requested length.
- Create first_three with slice(0, 3), middle with slice(4, 8),
  and from_seventh with slice(6, None).
- Preserve result widths 3, 4, 14; trim trailing spaces only
  in the printed view, not in the stored columns.
- Include assertions for the widths and these visible triples:
  ABCDEFGHIJ: ABC, EFGH, GHIJ
  HelloWorld: Hel, oWor, orld
  SAS-to-Python: SAS, to-P, -Python
- Print all columns and briefly explain the position adjustment.

Show code in this response. Do not create or run files, or read
existing
Python files.
```

SAS REFERENCE

WHY THESE DETAILS HELP

- 1 Translate the rule**
The prompt shows how SAS start and length become Python start and stop.
- 2 Separate value and display**
Preserve stored widths. Trim only the printed view so a neat display does not change the data.
- 3 Check every slice**
Expected triples cover the beginning, middle, and remainder. Each expression gets a concrete check.

CHECK IT YOURSELF

Run the code. Compare all nine visible strings and stored widths 3, 4, 14. Row 2 ends with orld; row 3 ends with -Python.

Make position rules explicit, then check several outputs rather than trusting a familiar-looking code pattern.